

App Flow & UI-UX Flow Document

Navigation, user journeys, screen states, and interaction design for the public site and service portal

This document describes how users move through ElkaTech Launchpad and what they see at every step: the public marketing site, the authentication journey, and the role-specific portal experiences for customers, engineers, and admins. It covers the design system, navigation model, screen-by-screen behavior, empty/error/loading/blocked states, responsive and accessibility requirements, and the full route map — detailed enough to build or QA each screen without reverse-engineering the code.

Document	App Flow / UI-UX Flow Document
Product	ElkaTech Launchpad
Version	1.0
Date	June 10, 2026
Status	Approved baseline — reflects the shipped UI
Owner	ElkaTech Design / Frontend
Audience	Designers, frontend engineers, QA, stakeholders
Classification	Internal / Confidential
Source of truth	Repository README.md and apps/web source

Contents

1. UX Overview	4
2. Design Principles	4
2.1 Industrial premium feel	4
2.2 Graphite / ivory / copper design system	4
2.3 Responsive layout	4
2.4 Dark / light theme support	5
2.5 Stable animations	5
2.6 Accessibility	5
3. Public Website Flow	5
3.1 Landing page (/)	5
3.2 Product-category pages	5
3.3 Product detail / product cards	5
3.4 Contact / service CTA	6
4. Authentication Flow	6
4.1 Signup (/signup)	6
4.2 Login (/login)	6
4.3 Google sign-in	6
4.4 Firebase session bridge	7
4.5 Password reset	7
4.6 Email verification notice	7
5. Customer Portal Flow	7
5.1 Pending approval state	7
5.2 Approved customer state	7
5.3 Create service request (/app/requests/new)	7
5.4 View request list (/app/requests)	8
5.5 Request detail, conversation, and status visibility	8
6. Engineer Flow	8
6.1 Queue (/app/queue)	8
6.2 Claim request	8

6.3 View assigned request	9
6.4 Reply to customer vs. internal note	9
6.5 Update status	9
7. Admin Flow	9
7.1 Dashboard (/app/admin)	9
7.2 User approvals and management (/app/users)	9
7.3 Staff invitation	10
7.4 Assign / reassign engineer	10
7.5 Health dashboard, suspend/reactivate	10
8. Request Detail UX	10
9. Sidebar / Navigation Flow	10
10. Theme Switching Flow	11
11. UI States	11
11.1 Empty states	11
11.2 Error states	11
11.3 Loading states	12
11.4 Permission-blocked states	12
12. Mobile / Responsive Behavior	12
13. Accessibility Requirements	12
14. UI Component Inventory	13
15. Full Route Map	13
15.1 Public and auth routes	14
15.2 Protected portal routes	14
16. User-Flow Diagrams (Mermaid)	14
17. Screen-by-Screen Breakdown	15
18. UX Risks and Improvements	16
19. Future UX Enhancements	16
20. Assumptions and Open Questions	17

1. UX Overview

One React application, two worlds. The public world sells the brand: cinematic, dark-capable, motion-polished marketing and catalog pages. The portal world runs the service business: a calm, matte, information-dense workspace shared by customers, engineers, and admins, differing only in which routes and actions each role sees.

- Public visitors never hit a login wall; the portal is reached deliberately via sign-in or service CTAs.
- The portal is one shell (collapsible sidebar + content area) for all roles; role determines navigation items and available actions, never a different design language.
- Status is always legible: requests carry status badges everywhere, approval state is explained with dedicated cards, and service health is visualized for admins.
- Blocked is a designed state, not an error: pending approval, unverified email, and permission walls each have purposeful UI.

2. Design Principles

2.1 Industrial premium feel

The brand sells heavy industrial machines; the UI mirrors that: matte surfaces, restrained color, precise typography, and motion that feels engineered rather than playful. Decorative depth comes from `ProductPageBackground`'s theme-aware radial gradients, grid texture, and subtle particles — never from loud gradients.

2.2 Graphite / ivory / copper design system

- Portal surfaces use the unified token set: `lp-card` surfaces, `--lp-panel` backgrounds, `--lp-accent` copper, `--lp-line` hairlines.
- Cards share one matte surface treatment with subtle borders and rounded-2xl / rounded-xl radii; no bright blue buttons, badges, or glows remain in the portal.
- The public landing keeps its deeper navy/steel atmosphere for cinematic contrast; tokens live in `src/index.css` and `tailwind.config.ts`, and new UI must reuse existing tokens rather than invent one-off colors.
- Status badge palette — new/open: muted copper · triaged/pending: muted amber · resolved/approved/healthy: muted green · suspended/rejected/down: muted red · role chips: graphite neutral with copper accent.

2.3 Responsive layout

Every page works from phone to desktop. The landing sections reflow to single column; category specs tables scroll or stack; the portal sidebar collapses to an icon rail with tooltips.

2.4 Dark / light theme support

Three modes — light, dark, system — persisted under the localStorage key elkatech-ui-theme and applied as a root class before content paints, so reloads never flash the wrong theme. SiteHeader exposes light/dark variants so dark hero sections and light category pages both keep readable contrast.

2.5 Stable animations

- StableReveal is the only entrance primitive outside the hero: opacity + subtle blur, never transform/scale/layout animation, so cards never shift on refresh or differ across browsers.
- Hero and IntroAnimation may use Framer Motion expressively.
- Hover polish is limited to borders, shadows, background gradients, icon glow, and small in-button arrow motion.
- prefers-reduced-motion is respected throughout.

2.6 Accessibility

Interactive components are built on shadcn/Radix primitives (focus management, ARIA, keyboard behavior come from the primitives); both themes maintain readable contrast; reduced motion is honored. See section 13 for the requirement list.

3. Public Website Flow

3.1 Landing page (/)

1. IntroAnimation: branded loader on first load.
2. HeroSection: cinematic headline motion, primary CTAs.
3. Section flow: about → work → brands → infrastructure → why-us → contact → footer, each revealed with StableReveal.
4. SiteHeader stays fixed with scroll-aware glass states and theme-appropriate variant; navigation links to category pages and login.

3.2 Product-category pages

Seven routes (solvent, UV, laser cutting, lamination, desktop UV, inkjet, flatbed UV) share the ProductCategoryPage layout: title and intro, then a product list rendered from local component data.

3.3 Product detail / product cards

- Image carousel per product (assets from apps/web/public/images).
- Specification table (label/value rows).
- Display price (price field) and brochure link when available.
- Highlights list; theme-aware decorative background behind the page.

3.4 Contact / service CTA

Every category page exposes service-request entry points and the landing page ends in a contact section; CTAs route into the portal (login/signup if unauthenticated, then `/app/requests/new`).

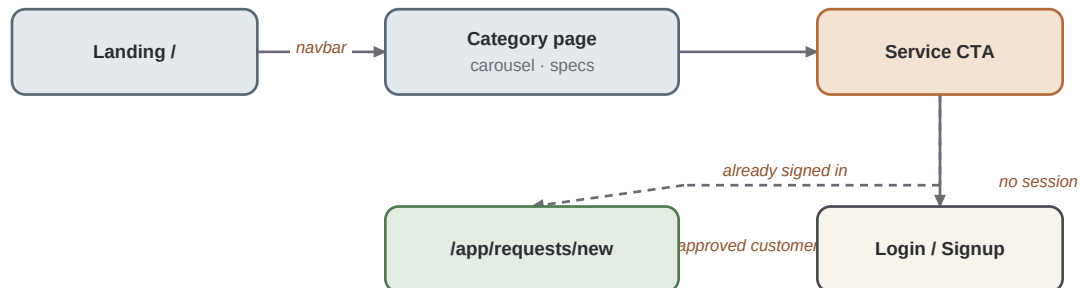


Figure 1 — Public-to-portal conversion path.

4. Authentication Flow

4.1 Signup (/signup)

1. User chooses email/password or Google; the same page also accepts staff invites (invite token path).
2. Email/password: Firebase `createUserWithEmailAndPassword`, then `sendEmailVerification`.
3. The Firebase ID token is exchanged at `POST /api/auth/firebase/session`; the local user is created with role `customer`, approval status `pending_approval`.
4. Session and CSRF cookies are set; the session cache is seeded synchronously so the first navigation into `/app` works in one click.
5. User lands in the portal in the pending-approval state.

4.2 Login (/login)

- Firebase `signInWithEmailAndPassword` → ID token → session bridge; legacy backend login remains for pre-Firebase accounts.
- Pending users can sign in and browse; the new-request form is replaced by the approval card.

4.3 Google sign-in

`signInWithPopup` with the Google provider, then the same session bridge and approval rules. One click is sufficient: the session query cache is set synchronously before navigation (the historical double-click race is fixed). A legacy server-side OAuth flow (`/api/auth/google/start` → callback) remains available.

4.4 Firebase session bridge

```
flowchart LR
    A[Firebase sign-in\npopup or password] --> B[ID token]
    B --> C[POST /api/auth/firebase/session]
    C --> D[verify via Admin SDK\ncreate/sync local user]
    D --> E[Set session + CSRF cookies]
    E --> F[seed session cache -> /app]
```

Listing 1 — Session bridge, Mermaid source.

4.5 Password reset

- /forgot-password: with Firebase configured, sendPasswordResetEmail handles delivery; otherwise the legacy backend flow (request email → /reset-password completion) is used.
- Signed-in users can also trigger a reset for their own address from My Account (rate-limited, constant response).

4.6 Email verification notice

Firebase sends verification mail for Firebase accounts; the legacy /verify-email route still completes legacy tokens. Inside the portal, signed-in-but-unverified users see the friendly VerifyEmailNotice banner; request creation requires a verified email.

5. Customer Portal Flow

5.1 Pending approval state

- /app/requests/new shows the polished ApprovalStateCard instead of the form (also styled for rejected and suspended).
- /app/requests shows an explanatory banner.
- Server enforcement mirrors the UI: POST /api/requests returns USER_PENDING_APPROVAL / USER_REJECTED / USER_SUSPENDED.

5.2 Approved customer state

Full customer navigation: My requests, New request, My Account. The sidebar profile block shows display name and email.

5.3 Create service request (/app/requests/new)

1. Select product from the catalog-backed selector (seeded database catalog).
2. Fill subject, description, contact phone, site location, optional serial number; pick priority (low / normal / high / urgent).

3. Submit → request created with status new and a unique request number; confirmation email follows; UI navigates to the request detail.

5.4 View request list (/app/requests)

- Customer sees own requests with StatusBadge per row and key metadata; staff visiting the same route see their relevant scope.
- Empty state invites creating the first request.

5.5 Request detail, conversation, and status visibility

Detail shows the compact summary card, the chat-style conversation (customer-visible messages only — internal notes are never delivered to customers), the composer, the assignee ('Assigned to *name*' once claimed), and the activity timeline. Status changes appear as timeline events and badge updates; section 8 details the layout.

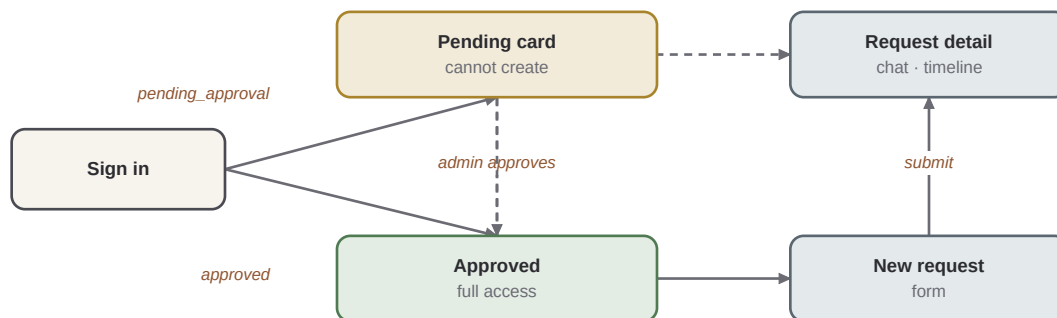


Figure 2 — Customer journey through approval and request creation.

6. Engineer Flow

6.1 Queue (/app/queue)

Engineers and admins see open and assigned requests (GET /api/requests?scope=queue) with status badges, priority pills, and assignment state.

6.2 Claim request

- Primary copper Claim button on unassigned requests (workflow panel).
- Claim sets the engineer as assignee and writes request_claimed history; if someone else claimed first, a 409 with a friendly message is surfaced — no silent steal.

6.3 View assigned request

/app/requests lists the engineer's assigned work; request detail shows everything the customer sees plus internal notes and workflow controls.

6.4 Reply to customer vs. internal note

- The composer's visibility selector switches between customer-visible reply and internal note.
- Internal notes render with an 'Internal note' badge; staff role chips (Admin / Engineer) appear as small mono pills on messages.

6.5 Update status

Status actions render in a two-column grid showing only valid forward transitions for the current state and role (getAllowedStatuses); button copy is context-aware (Mark triaged, Resume work, Reopen request...). Invalid moves are rejected server-side with 400/403 even if attempted directly.

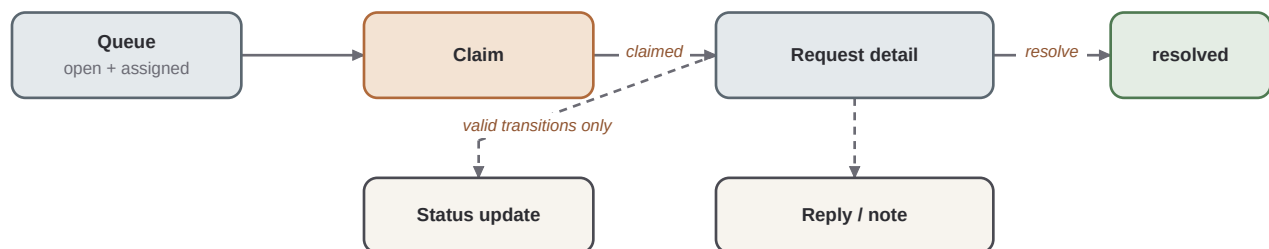


Figure 3 — Engineer working loop.

7. Admin Flow

7.1 Dashboard (/app/admin)

- Stat cards: approval-status counts and service-request load.
- Recent activity and recent signups panels.
- Live heartbeats: healthy / degraded / down per service with latency, refreshing every 60 seconds.

7.2 User approvals and management (/app/users)

- User list with role, approval status, account origin, and verification state; approval actions: approve, reject, suspend, reactivate.
- Remove user: permanent deletion with an explicit warning modal; the Remove button never shows for admin accounts; on success the user disappears from the list immediately.

7.3 Staff invitation

Invite engineer/admin by email; invited accounts arrive approved with the invited role, and an account-added email (SES template) is sent. Self-signups remain locked to the customer role; only staff-managed accounts are eligible for promotion.

7.4 Assign / reassign engineer

On a request's workflow panel, admins use the inline [Select engineer] [Reassign] row. Assigning writes request_assigned; changing the assignee writes request_reassigned with previous/new engineer metadata; assigning the same engineer no-ops to keep history clean.

7.5 Health dashboard, suspend/reactivate

Heartbeats live on the dashboard (see 7.1); suspending a user drops their sessions and blocks request creation until reactivated.

8. Request Detail UX

The request detail page is the platform's richest screen, structured as a two-column layout (stacking on mobile):

Region	Content and behavior
Summary card	Request number, StatusBadge, priority pill, subject, product, description, and a meta strip with created / updated / assignee — deliberately compact, no hero block.
Conversation	Real chat: own messages right-aligned in a copper-tinted bubble with a 'You' label; others left-aligned in graphite bubbles with sender name (fallback: email, then role label). Role chips (Admin / Engineer) and 'Internal note' badges render as small mono pills.
Composer	Textarea sits flush inside the conversation card; a thin divider row holds the visibility selector and the Send pill; Send disables on empty input.
Details sidebar	A single card of DetailRow items (label + value with hairline dividers), including 'Assigned to' resolving to You / display name / email / Unassigned.
Workflow panel	Assignment-state pill on top; primary copper Claim button when unassigned; status actions in a two-column grid (valid targets only, context-aware copy); admin Reassign as an inline select+button row; Archive isolated under a divider with caution styling.
Activity timeline	Vertical line with copper dots, newest first, compact rows, an 'N events' count badge, and 'Show all activity (X more)' / 'Show less' (default cap 5).

9. Sidebar / Navigation Flow

- PortalShell wraps every /app route: collapsible sidebar, content area, toasts.
- Navigation by role — customer: Requests, New request; engineer: adds Queue; admin: adds Admin dashboard and Users.

- My Account sits directly above Log out; both support expanded and collapsed widths with active-state styling and tooltips.
- Profile block shows display name + email (role tag removed for declutter); theme row reads as a compact '[icon] System >' selector without a redundant label.
- Collapsed mode is an icon rail with tooltips; expanded mode shows labels.

10. Theme Switching Flow

```
sequenceDiagram
    participant User
    participant Toggle as ThemeToggle
    participant Provider as ThemeProvider
    participant Storage as localStorage
    participant Root as documentElement
    User->>Toggle: Choose light/dark/system
    Toggle->>Provider: setTheme(theme)
    Provider->>Storage: Persist elkatech-ui-theme
    Provider->>Root: Replace light/dark class
    Root-->>User: Updated themed UI
```

Listing 2 — Theme toggle flow (README sequence diagram).

- System mode tracks the OS preference; explicit choices persist across sessions.
- Both the public SiteHeader and the portal theme row drive the same provider; tokens swap via the root class so every component re-themes consistently.

11. UI States

11.1 Empty states

Screen	Empty state
Requests list	Friendly prompt to create the first request (customers) / no assigned work message (staff)
Queue	No open requests — calm confirmation, not an error
Conversation	Request description anchors the thread; composer invites the first message
Users (filtered)	No matches for the active filter
Activity timeline	Always has at least request_created

11.2 Error states

- Mutation failures surface as toasts with the server's message (e.g. 'Invalid workflow transition.', claim conflict).
- 403s render permission messaging; approval-specific codes render the dedicated cards/banners rather than raw errors.

- Unknown routes fall through the SPA to a not-found view; failed loads offer retry via TanStack Query refetch.

11.3 Loading states

- Initial app load: branded IntroAnimation.
- Portal data: skeleton/placeholder surfaces in cards while queries resolve; ProtectedRoute resolves the session before rendering gated content.
- Buttons disable during pending mutations (Send disabled on empty/pending).

11.4 Permission-blocked states

Situation	Experience
Pending approval	ApprovalStateCard on /app/requests/new + banner on /app/requests; cannot submit
Rejected / suspended	Same card pattern with state-specific copy
Unverified email	VerifyEmailNotice banner; creation blocked until verified
Role-gated route	ProtectedRoute redirects to the user's safe default (/app/requests)
Internal notes	Never rendered for customers — absent, not masked

12. Mobile / Responsive Behavior

- Landing and category pages reflow to single-column; carousels remain swipeable; spec tables stay readable at narrow widths.
- Portal sidebar collapses to the icon rail; tooltips replace labels; the request detail's two-column layout stacks (summary → conversation → workflow/details).
- Touch targets follow Radix primitive sizing; composer and status grids remain usable on phones.
- Theme and navigation are reachable at every breakpoint.

13. Accessibility Requirements

- Respect prefers-reduced-motion for all entrance and hover animation (StableReveal honors it by design).
- Maintain readable contrast in both themes, including the navbar's light/dark variants over hero imagery.
- Keyboard operability and ARIA semantics via Radix primitives for dialogs, selects, tooltips, and toasts.
- Focus visibility on interactive elements; logical heading structure per page.
- Status communicated by text + badge, never color alone (badges carry labels).
- Form fields labelled; validation errors announced inline.

Assumption / open question. A formal WCAG conformance target (e.g. 2.1 AA) and an audit are not documented in the README; adopting AA as the explicit bar is recommended.

14. UI Component Inventory

Component	Responsibility
HeroSection	Cinematic landing hero and headline motion
SiteHeader	Shared responsive navbar; dark/light variants; scroll-aware glass states
ProductCategoryPage	Shared category layout: carousel, specs, product CTA composition
ProductPageBackground	Theme-aware radial depth, grid texture, decorative particles
StableReveal	Fade/blur reveal wrapper; no transform-based entrance motion
ThemeProvider / ThemeToggle	Persisted system/light/dark theme control
IntroAnimation	Initial branded loader animation
ProtectedRoute	Session gate plus role-based route fallback
PortalShell	Collapsible sidebar shell with theme selector, profile block, My Account, logout
StatusBadge	Request status presentation (muted palette per state)
ApprovalStateCard	Pending / rejected / suspended state card for blocked customers
VerifyEmailNotice	Banner for signed-in users with unverified email
AdminDashboardPage	Approval counts, request load, recent activity, live heartbeats
DetailRow	Label/value rows with hairline dividers in the request sidebar
shadcn/Radix primitives	Buttons, dialogs, selects, toasts, tooltips, form controls

15. Full Route Map

15.1 Public and auth routes

Route	Purpose	Access
/	Landing page	Public
/solvent-printers	Solvent printer category	Public
/uv-printers	UV printer category	Public
/laser-cutting-machines	Laser cutting category	Public
/lamination-machines	Lamination category	Public
/desktop-uv-printer	Desktop UV category	Public
/inkjet-printer	Inkjet printer category	Public
/inject-printer	Redirect → /inkjet-printer	Public
/flatbed-uv-printer	Flatbed UV category	Public
/login	Sign in	Public
/signup	Signup and invite acceptance	Public
/forgot-password	Password reset request	Public
/reset-password	Password reset completion	Public
/verify-email	Email verification (legacy tokens)	Public

15.2 Protected portal routes

Route	Purpose	Access
/app	Redirects to /app/requests	Authenticated
/app/requests	Customer requests / assigned work	Authenticated
/app/requests/new	Create service request (approval-gated)	Authenticated
/app/requests/:requestId	Request detail: messages, history, workflow	Authenticated with request access
/app/queue	Open/assigned service queue	engineer, admin
/app/admin	Admin dashboard: approvals, load, heartbeats	admin
/app/users	User list, approvals, staff invitations	admin
/app/account	My Account: profile, password reset	All authenticated roles

16. User-Flow Diagrams (Mermaid)

Canonical Mermaid sources for the three role journeys; render in any Mermaid-capable tool.

```

flowchart TD
    A[Visitor] -->|Sign up| B[Identity]
    B -->|Email/password| C[Firebase create + verify email]
    B -->|Google popup| D[Firebase Google sign-in]
    C --> E[POST /api/auth/firebase/session]
    D --> E
    E --> F[Session + CSRF cookies]
    F --> G[approval_status]
    G -->|pending_approval| H[Pending card - cannot create]
    G -->|approved| I[Create request]
    H -->|Admin approves| I
    I --> J[Track status + converse]
    J -->|request.assigned email| J

```

Listing 3 — Customer journey.

```

flowchart TD
    Q[Queue /app/queue] -->|Claim| A[Already assigned?]
    A -->|Yes - 409| Q
    A -->|No| W[Assigned to me]
    W --> M[Reply or internal note]
    W --> S[Status update]
    S -->|in_progress| W
    S -->|waiting_for_customer| W
    S -->|resolved| R[Resolved]
    R -->|close| C[Closed]
    C -. admin reopen .-> Q

```

Listing 4 — Engineer journey.

```

flowchart TD
    D[Admin dashboard] --> U[Users page]
    U -->|Approve| AP[Customer can create requests]
    U -->|Reject / Suspend| BL[Blocked states]
    U -->|Reactivate| AP
    U -->|Invite staff| IV[Engineer/admin account + SES email]
    U -->|Remove user| RM[Permanent delete - admins protected]
    D --> H[Service heartbeats 60s]
    D --> RQ[Request load + recent activity]
    RQ --> AS[Assign / reassign engineer]

```

Listing 5 — Admin journey.

17. Screen-by-Screen Breakdown

Screen	Key elements	Primary actions
Landing (/)	Intro loader, hero, section stack, themed navbar, contact, footer	Navigate to categories; contact; login
Category page	Title + intro, product cards: carousel, specs, price, brochure, highlights; decorative background	Browse products; download brochure; service CTA
Login	Email/password form, Google button, links to signup/forgot-password	Sign in (one click to portal)
Signup	Email/password or Google; invite-token acceptance	Create account; accept invite

Screen	Key elements	Primary actions
Forgot / Reset password	Email entry; new-password form (legacy completion)	Request reset; set new password
Verify email	Legacy token completion view	Verify
Requests list (/app/requests)	Request rows with status badges; pending banner when applicable; empty state	Open detail; New request
New request (/app/requests/new)	Product selector, subject, description, phone, location, serial, priority — or ApprovalStateCard when blocked	Submit request
Request detail	Summary card, chat thread, composer, details sidebar, workflow panel, activity timeline	Reply/note; claim; status; assign (admin); archive
Queue (/app/queue)	Open + assigned requests with badges and priority pills	Open detail; claim
Admin dashboard (/app/admin)	Stat cards, recent activity/signups, heartbeat grid	Monitor; jump to users/requests
Users (/app/users)	User table: role, approval, origin, verification; invite form; action menus	Approve/reject/suspend/reactivate; invite; remove
My Account (/app/account)	Display name, email, role, approval status, origin, joined date, verification state; copper notice that email changes are admin-handled	Edit display name; password reset

18. UX Risks and Improvements

Risk / gap	Impact	Recommended improvement
Dual catalog sources	Public page and portal selector can disagree on a product	Single-source generation; until then, paired updates
Approval wait is passive for the customer	Uncertainty after signup	Set expectations on the pending card (e.g. 'typically within 1 business day'); notify on decision
No in-portal notifications	Email-only signals can be missed	Lightweight notification center / unread markers
No attachments in requests	Customers cannot share photos of faults	Add image attachments to the composer and creation form
Timeline cap of 5 may hide context	Long histories need an extra click	Acceptable; keep 'Show all' affordance prominent
Legacy + Firebase auth duality	Two paths to maintain and QA	Plan migration of legacy accounts to Firebase

19. Future UX Enhancements

- Request attachments (photos/video of machine faults).
- In-portal notification center with unread indicators.
- Saved machine profiles per customer (serials, locations) to prefill request forms.
- Searchable knowledge base / brochures hub inside the portal.

- Queue filters (priority, category, age) and saved views for engineers.
- Dashboard trends (resolution time, volume over time) for admins.
- Localization readiness if non-English markets are pursued.

20. Assumptions and Open Questions

- **Not-found behavior.** SPA fallback exists via Vercel rewrites; the precise 404 view design is implementation-defined and should be confirmed in QA.
- **Loading skeletons.** The README confirms loading polish (intro animation, stable reveals); the exact skeleton treatment per portal screen follows the implementation.
- **Archive semantics.** The workflow panel exposes Archive with caution styling and history records request_archived; the customer-facing visibility of archived requests should be documented explicitly.
- **WCAG target.** No formal conformance level is committed; AA recommended.
- **Contact section behavior.** The landing contact section's exact submission mechanics (form vs. links) are implementation-defined in the components.